

FPGA experiment report

Task: FPGA 2

Student 1: name: mahmood stitia ID: 214032682

Student 2: name: Saleh Khalil ID: 213310485

Part 1 - Status update

Fill in the following table according to your progress at the time of the submission.

Each row should refer to one of the modules or a simulation. For example, write FA under module name and refer to Limited_Incrementor.v and Limited_Incrementor_tb.v in that line.

The status column should be filled with one of the following:

- Done - If both are written and simulation passed.
- Partially done - If you started writing the module and simulation but it's not working properly.
- Not started - If you didn't get to that part of the task.

When the status is partially done, please use the notes/comments column to explain in a few more words what is the status of the module and what is the status of the simulation.

Fill here:

Module name/Step	status	notes/comments
Limited Incrementor	done	
Counter	done	

Issues:

In this section please describe the current issues you are facing.

- Per each module and simulation, if you receive any errors at the time of submission, please describe which errors and what they mean.

Fill here:

1. Limited Incrementor: No issues. The module compiles and simulates correctly.
2. Counter: No issues. The module compiles and simulates correctly.

Part 2 - Simulations and answers

In this section, please add the images for each simulation as described in the report. Remember to use annotations and explain in your own words what is shown in each simulation.

In some of the tasks in the report, there are also questions that you should answer here.

Make sure to include images of all simulations running at the time of submission, even if they are not correct.

1. Limited Incrementor

Description:

We implemented the Limited Incrementor (Lim_Inc) module, which performs an increment operation on a value a with carry-in ci , and to a limit L . The module uses a Conditional Sum Adder (CSA) that we previously implemented to compute $a + ci$. If the result is greater than or equal to L , the output sum is set to 0 and co is set to 1. Otherwise, sum equals the addition result and co is 0.

The parameter L is set to 10, and $N = \lceil \log_2(L) \rceil = 4$ bits.

Verilog code - Lim_Inc.v:

```
1 module Lim_Inc(a, ci, sum, co);
2     parameter L = 10;
3     localparam N = $clog2(L);
4
5     input [N-1:0] a;
6     input ci;
7     output [N-1:0] sum;
8     output co;
9
10    wire [N-1:0] inc_a;
11    wire csa_co;
12
13    CSA #(.N(N)) csa (
14        .a(a),
15        .b({N{1'b0}}),
16        .ci(ci),
17        .sum(inc_a),
18        .co(csa_co)
19    );
20
21    assign sum = ({csa_co, inc_a} < L) ? inc_a : {N{1'b0}};
22    assign co = ({csa_co, inc_a} < L) ? 1'b0 : 1'b1;
23 endmodule
```

Test-Bench description:

The testbench iterates over all values representable using 4 bits (i.e., 0 to 15), with both $c_i = 0$ and $c_i = 1$, driving each combination to the design under test. The expected output is verified according to three cases:

Case 1: $a_i < L - 1$ In this range, the carry-in has no effect on the result. The expected output will always be:

$$c_o = 0, \quad \text{sum} = a_i + c_i$$

Case 2: $a_i = L - 1$ This is the boundary condition. The output depends on the value of c_i , so both $c_i = 0$ and $c_i = 1$ are checked separately to ensure correct behavior at the edge of the valid range.

Case 3: $a_i \geq L$ In this range, the input equal or larger than L . So regardless of the value of c_i , the output is:

$$c_o = 1, \quad \text{sum} = 0$$

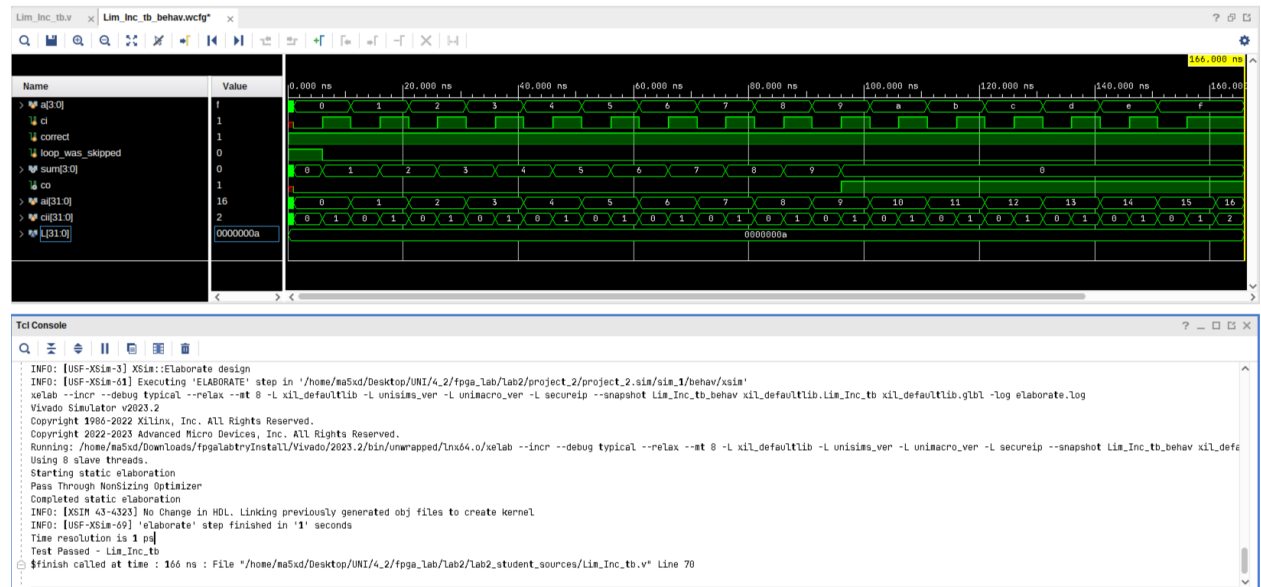
Test-Bench code - Lim_Inc_tb.v:

```
1 module Lim_Inc_tb();
2     reg [3:0] a;
3     reg ci, correct, loop_was_skipped;
4     wire [3:0] sum;
5     wire co;
6     integer ai, cii;
7     localparam L = 10;
8     Lim_Inc #(L) uut(
9         .a(a), .ci(ci), .sum(sum), .co(co)
10    );
11    initial begin
12        correct = 1;
13        loop_was_skipped = 1;
14        #1
15        for (ai = 0; ai < 16; ai = ai + 1) begin
16            for (cii = 0; cii <= 1; cii = cii + 1) begin
17                a = ai; ci = cii;
18                #5;
19                if (ai >= L)
20                    correct = correct & (sum == 0) & (co == 1);
21                else begin
22                    if (ai == L-1)
23                        correct = correct &
24                            (((sum==ai)&(co==0)&(cii==0)) |
25                             ((sum==0)&(co==1)&(cii==1)));
26                    else
27                        correct = correct &
28                            (sum == ai+cii) & (co == 0);
29                end
30                loop_was_skipped = 0;
31            end
32        end
33        #5
34        if (correct && ~loop_was_skipped)
35            $display("Test Passed - %m");
36        else
37            $display("Test Failed - %m");
38        $finish;
39    end
```

40 endmodule

Simulation waveform – full view:

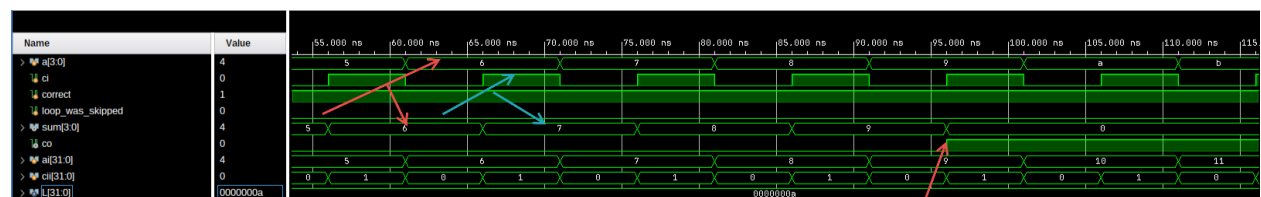
The figure below shows the complete simulation of the Limited Incrementor testbench. The signal `a[3:0]` increments from 0 to 15 (hex f), with `ci` toggling between 0 and 1 for each value. The `sum[3:0]` output correctly follows `a + ci` for values below $L = 10$, and resets to 0 for values $\geq L$. The correct flag remains 1 throughout, and the TCL console shows “Test Passed”.



Simulation waveform - (overflow region):

This zoomed-in view focuses on the region where `a` transitions from 5 to beyond 9. The colored arrows highlight key behaviors:

- **Red arrows:** Show the sum output when `ci` = 0. For example, when `a` = 5 and `ci` = 0, `sum` = 5.
- **Blue arrows:** Show the sum output when `ci` = 1. For instance, when `a` = 6 and `ci` = 1, `sum` = 7 (i.e., 6+1).
- **Red arrow at `a` = 9:** When `a` = 9 and `ci` = 1, the result $9+1 = 10 \geq L$, so `sum` resets to 0 and `co` becomes 1. This confirms the overflow detection works correctly at the boundary.



2. Counter

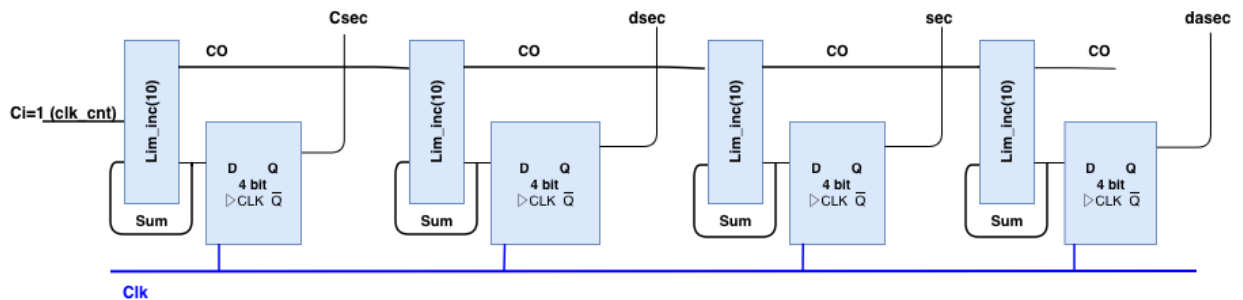
Question (a): The output `time_reading[15:0]` is composed of 4 digits:

$\{tens_seconds[3:0], ones_seconds[3:0], tens_centiseconds[3:0], ones_centiseconds[3:0]\}$

Each digit ranges from 0 to 9, so the counter represents values from 00:00 to 99:99. Therefore the time range is

0 to 99.99 seconds.

Question (b): To count centiseconds from a 100 MHz clock, we first use a clock-cycle counter (`clk_cnt`) that counts from 0 to $CLK_FREQ/100 - 1 = 999,999$. When `clk_cnt` reaches its maximum value, it generates a `tick` signal. This tick signal is fed as the carry-in (`ci`) to a `Lim_Inc(10)` instance whose input `a` is connected to a 4-bit register holding the current centisecond digit. The `sum` output of `Lim_Inc` is wired back to the register through an always `@(posedge clk)` block. When the digit reaches 9 and receives a tick, it rolls over to 0 and produces a carry-out (`co`) that serves as the carry-in for the next digit's `Lim_Inc` instance.



Verilog code - Counter.v:

```
1 module Counter(clk, init_regs, count_enabled,  
2               count_sample, show_sample, time_reading);  
3     parameter CLK_FREQ = 100000000; // in Hz  
4     input  clk, init_regs, count_enabled,  
5           count_sample, show_sample;  
6     output [15:0] time_reading;  
7     reg [$clog2(CLK_FREQ/100)-1:0] clk_cnt;  
8     reg [3:0] ones_centiseconds;  
9     reg [3:0] tens_centiseconds;  
10    reg [3:0] ones_seconds;  
11    reg [3:0] tens_seconds;  
12    localparam TICKS_PER_CSEC = CLK_FREQ / 100;  
13    wire tick = (clk_cnt == TICKS_PER_CSEC - 1)  
14              & count_enabled;  
15    wire co0, co1, co2, co3;  
16    wire [3:0] ones_cs, tens_cs, ones_s, tens_s;  
17    reg [15:0] sample_reg;  
18    Lim_Inc #(.L(10)) ones_cs_inc(  
19        .a(ones_centiseconds), .ci(tick),  
20        .sum(ones_cs), .co(co0));  
21    Lim_Inc #(.L(10)) tens_cs_inc(  
22        .a(tens_centiseconds), .ci(co0),  
23        .sum(tens_cs), .co(co1));  
24    Lim_Inc #(.L(10)) ones_s_inc(  
25        .a(ones_seconds), .ci(co1),  
26        .sum(ones_s), .co(co2));  
27    Lim_Inc #(.L(10)) tens_s_inc(  
28        .a(tens_seconds), .ci(co2),  
29        .sum(tens_s), .co(co3));  
30    always @(posedge clk) begin  
31        if (init_regs) begin  
32            clk_cnt <= 0;  
33            ones_centiseconds <= 0;  
34            tens_centiseconds <= 0;  
35            ones_seconds <= 0;  
36            tens_seconds <= 0;  
37            sample_reg <= 0;  
38        end else begin  
39            if (clk_cnt == TICKS_PER_CSEC - 1)  
40                clk_cnt <= 0;  
41            else  
42                clk_cnt <= clk_cnt + 1;  
43            ones_centiseconds <= ones_cs;  
44            tens_centiseconds <= tens_cs;  
45            ones_seconds <= ones_s;  
46            tens_seconds <= tens_s;  
47            if (count_sample)  
48                sample_reg <= {tens_seconds,  
49                               ones_seconds,  
50                               tens_centiseconds,  
51                               ones_centiseconds};  
52        end  
53    end  
54    assign time_reading = show_sample ? sample_reg  
55        : {tens_seconds, ones_seconds,  
56           tens_centiseconds, ones_centiseconds};  
57 endmodule
```

Test-Bench code - Counter_tb.v:

```
1 module Counter_tb();
2     reg clk, init_regs, count_enabled,
3         count_sample, show_sample,
4         correct, loop_was_skipped;
5     wire [15:0] time_reading;
6     wire [3:0] tens_seconds_wire;
7     wire [3:0] ones_seconds_wire;
8     wire [3:0] tens_centiseconds_wire;
9     wire [3:0] ones_centiseconds_wire;
10    integer ts, os, tc, oc, sync;
11
12    Counter uut (
13        .clk(clk), .init_regs(init_regs),
14        .count_enabled(count_enabled),
15        .count_sample(count_sample),
16        .show_sample(show_sample),
17        .time_reading(time_reading));
18
19    assign tens_seconds_wire      = time_reading[15:12];
20    assign ones_seconds_wire      = time_reading[11:8];
21    assign tens_centiseconds_wire = time_reading[7:4];
22    assign ones_centiseconds_wire = time_reading[3:0];
23
24    initial begin
25        #1
26        sync = 0; count_sample = 0;
27        show_sample = 0; correct = 1;
28        loop_was_skipped = 1;
29        clk = 1; init_regs = 1;
30        count_enabled = 0;
31        #20
32        init_regs = 0; count_enabled = 1;
33        for (ts=0; ts<1; ts=ts+1) begin
34            for (os=0; os<2; os=os+1) begin
35                for (tc=0; tc<10; tc=tc+1) begin
36                    for (oc=0; oc<10; oc=oc+1) begin
37                        #(9999999 + sync)
38                        sync = sync | 1;
39                        loop_was_skipped = 0;
40                        if (ts==0&&os==0&&tc==5&&oc==0)
41                            count_sample = 1;
42                        else count_sample = 0;
43                        if (ts==0&&os==1&&tc==0&&oc==0) begin
44                            show_sample = 1;
45                            correct = correct &&
46                                (tens_seconds_wire      == 0) &&
47                                (ones_seconds_wire      == 0) &&
48                                (tens_centiseconds_wire == 5) &&
49                                (ones_centiseconds_wire == 0);
50                        end else begin
51                            show_sample = 0;
52                            correct = correct &&
53                                (tens_seconds_wire      == ts) &&
54                                (ones_seconds_wire      == os) &&
55                                (tens_centiseconds_wire == tc) &&
56                                (ones_centiseconds_wire == oc);
57                        end
58                    end
59                end
60            end
61        end
62    end
```

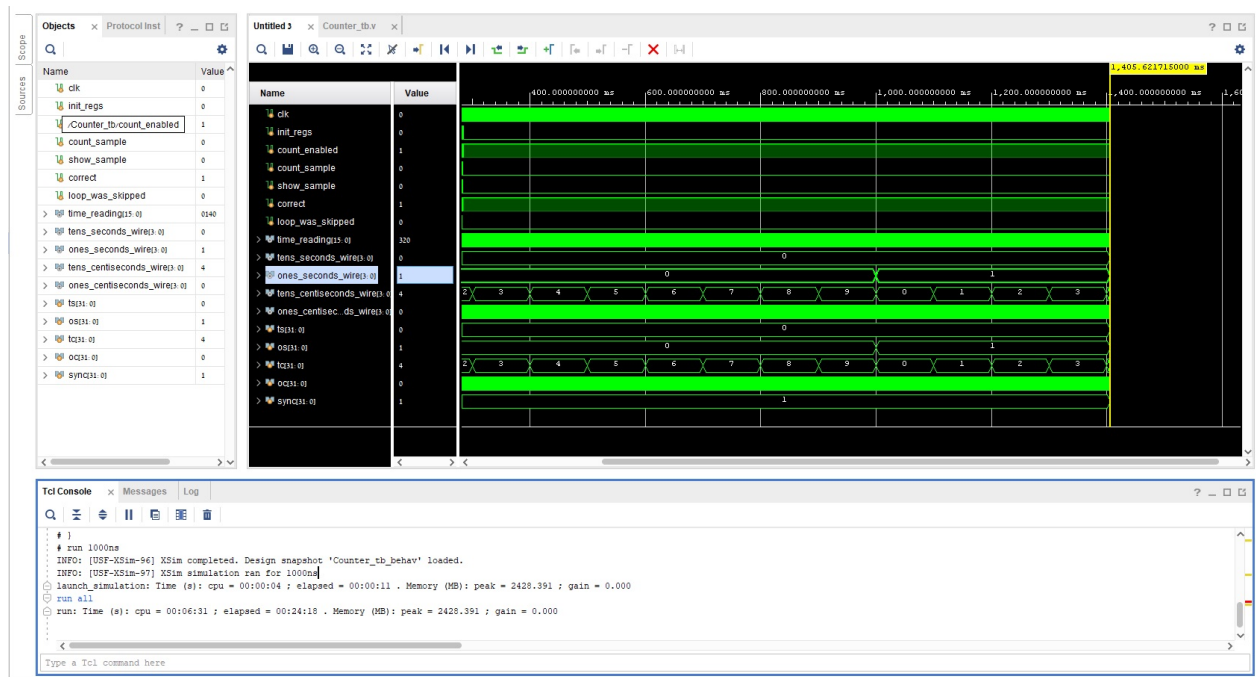
```

58         end
59     end
60 end
61 end
62 #5
63 if (correct && ~loop_was_skipped)
64     $display("Test Passed - %m");
65 else
66     $display("Test Failed - %m");
67 $finish;
68 end
69 always #5 clk = ~clk;
70 endmodule

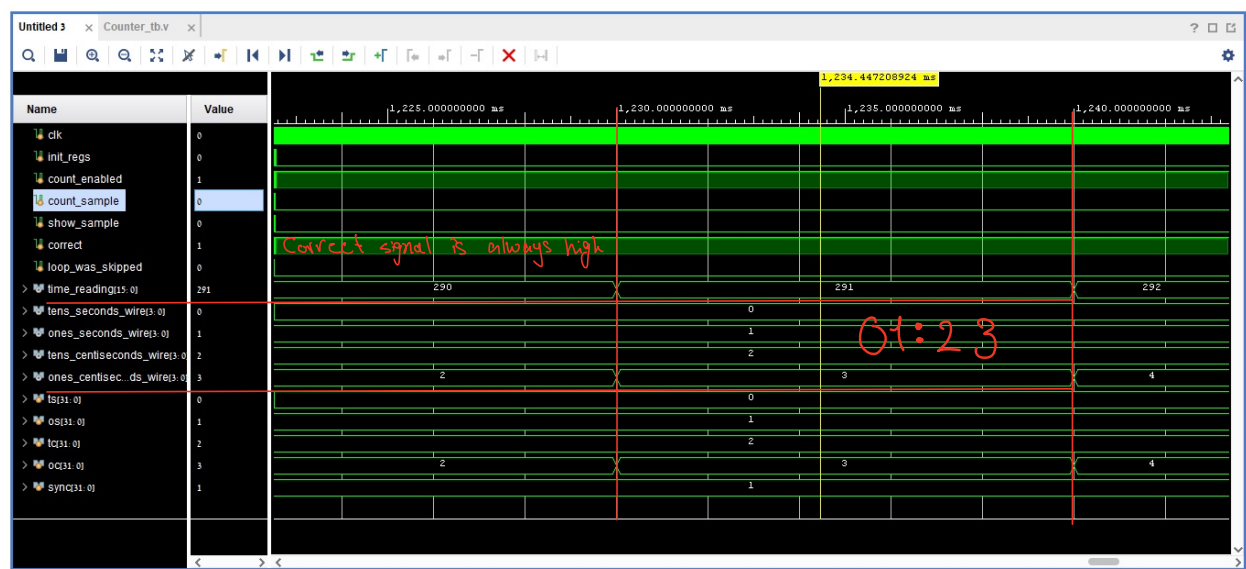
```

Simulation waveform:

The figure below shows passing tb of the counter simulation over 1.405ms



Simulation Waveform — Zoomed-In View



The correct signal remains continuously high throughout the simulation, confirming that all test cases produced the expected outputs. At the cursor position of approximately 1,234 ms of simulated time, the digit outputs tens_seconds, ones_seconds, tens_centiseconds, and ones_centiseconds read 0, 1, 2, and 3 respectively, encoding the time 01:23. This corresponds to a time_reading of 291 (decimal encoding of the 16 bits time_reading signal). The digit outputs increment sequentially and in sync, consistent with the expected counter behavior.